

STUDY NOTES · DEVELOPER JOURNAL

Cheap Flight Checker

How I built a daily flight price checker with Django, Celery, Redis and a Telegram bot. Step by step, with every mistake and fix along the way.

Bishkek (BSZ) ⇄ Tokyo (TYO)

Python 3.14 · Django 6.1 · DRF 3.18 · PostgreSQL 17 · Redis 7 · Celery 5.6 · Celery Beat · Docker Compose · Travelpayouts API · Telegram Bot API

Oct 22 – Oct 28, 2026

github.com/tilenur/cheap-flight-checker

Contents

PART 1: STUDY NOTES

1 Project overview — what it does, the big picture

2 Tech stack & project structure — tools and files

3 Step 1: Setup — venv, .env, Docker, settings

4 Step 2: Models & admin — Route, FlightOffer

5 Step 3: Fetching prices — services.py, fetch_flights

6 Step 4: Celery & Beat — background tasks, 9:00 schedule

7 Step 5: API & Redis cache — DRF, cache_page

8 Step 6: Telegram bot — sending the daily message

9 Step 7: Timezones & security — UTC vs Bishkek, hiding the token

10 Concepts glossary — quick definitions

11 Lessons learned & next ideas

PART 2: CHEAT SHEET

12 Terminal cheat sheet — venv, Django, Docker, Redis, Celery, Git, Telegram, Mac

13 Shell snippets & daily checklist — Python shell, troubleshooting

💡 **How to read these notes**

Each step has: a **goal**, the **key code**, the **new concepts** (blue boxes), and a table of the **mistakes I made and how I fixed them**. The mistakes are the most useful part, because they're the things I will probably meet again.

1 Project overview

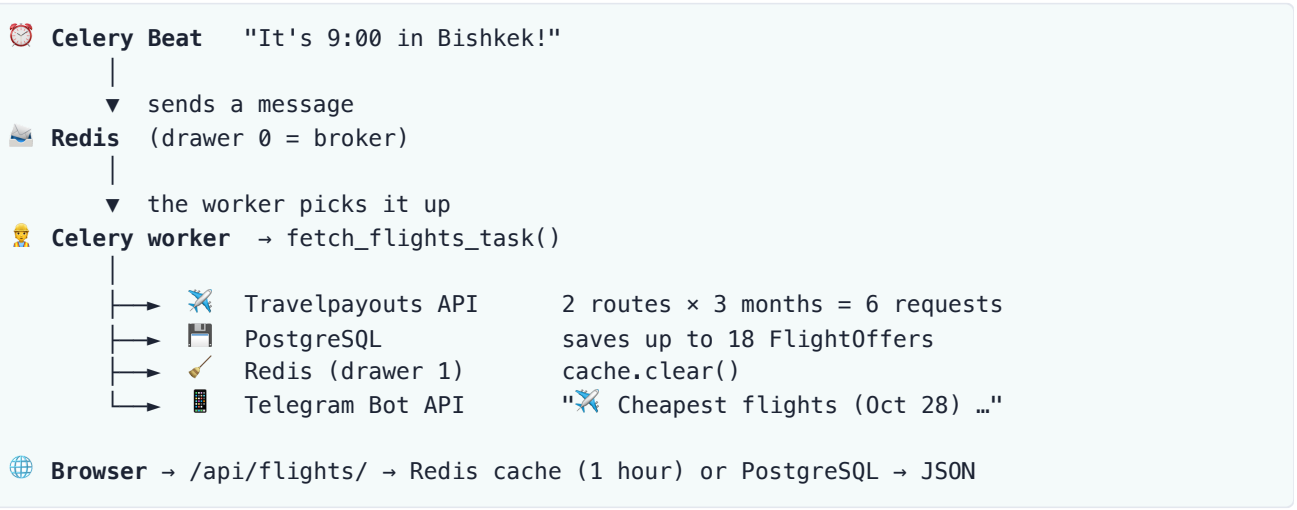
A small backend project to practise **Redis** and **Celery**. Every morning it finds the cheapest one-way flights between Bishkek and Tokyo, saves them, shows them in an API, and sends me a Telegram message.

2	3	3	9:00
directions BSZ→TYO, TYO→BSZ	months ahead starting next month	cheapest flights per month	every day Bishkek time

The rules we agreed on

- **One-way** flights in **both directions**: Bishkek → Tokyo and Tokyo → Bishkek.
- For each of the **next 3 months** (from next month), keep the **3 cheapest** flights with their dates.
- **Max 1 stop**. Show the stop info, but don't filter by the transfer country.
- Prices in **USD**. Data from the **Travelpayouts** (Aviasales) Data API v3.
- Results in a **DRF API** + a daily **Telegram** message. (The first plan was Gmail, but I switched to Telegram because email felt like too much spam.)

The big picture



What a morning looks like

- **09:00** Beat puts `fetch_flights_task` into Redis.
- **09:00** The worker takes it and asks Travelpayouts for 6 lists of flights.
- **09:00** It keeps the 3 cheapest (max 1 stop) per month and saves them.
- **09:00** It clears the API cache, so the API shows fresh prices.
- **09:01** It builds the message and sends it to my Telegram. 📱
- **later** When I open `/api/flights/`, the first request goes to PostgreSQL, and then Redis keeps a copy for 1 hour.

The Telegram message

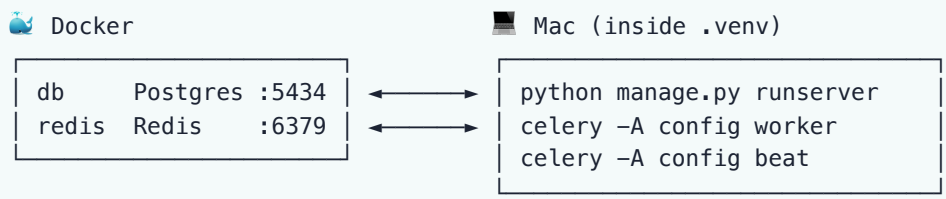
> ✈️ Cheapest flights (Oct 28)
>
> **BSZ → TYO**
> Nov 22 $405 CZ 1 stop Book
> Nov 24 $415 CZ 1 stop Book
> Nov 30 $402 CA 1 stop Book
>
> Dec 08 $401 CZ 1 stop Book
> …
>
> **TYO → BSZ**
> Nov 05 $432 CA 1 stop Book
> …

Each route is a box (`<blockquote>`), with an empty line between months. "Book" is a short link that opens Aviasales.

2 Tech stack & project structure

TOOL	WHAT IT IS	WHY WE USED IT
Django	Python web framework	Models, admin, settings, commands. The base of everything.
Django REST Framework	API toolkit for Django	Serializers + `ListAPIView` for `/api/flights/` .
PostgreSQL 17	Database	Saves routes and flight offers. Runs in Docker.
Redis 7	Very fast memory storage	**2 jobs:** Celery broker (drawer 0) and API cache (drawer 1). Runs in Docker.
Celery	Background task runner	Runs the fetch + Telegram work outside the web server.
Celery Beat	Scheduler (a clock)	Starts the task every day at 9:00 Bishkek time.
Docker Compose	Runs programs in containers	Postgres + Redis with one command, nothing installed on the Mac.
python-dotenv	Reads a `.env` file	Keeps secrets (tokens, passwords) out of the code.
requests	HTTP library	Calls the Travelpayouts and Telegram APIs.
Travelpayouts	Aviasales Data API v3	Free flight prices (`prices_for_dates`).
Telegram Bot API	Lets a program control a bot	Sends me the daily message (`sendMessage`).

Where things run



Project structure

```
cheap_flight_checker/
├── .env                        # secrets (NOT in git)
├── .gitignore
├── docker-compose.yml          # Postgres + Redis
├── requirements.txt            # pip freeze
├── manage.py
├── config/                     # the Django project
│   ├── __init__.py             # loads the Celery app
│   ├── celery.py               # creates the Celery app
│   ├── settings.py             # DB, Celery, Beat, cache, timezone
│   └── urls.py                 # admin/ + api/
└── flights/                    # our app
    ├── models.py               # Route, FlightOffer
    ├── admin.py
    ├── services.py             # ⭐ all the real work
    ├── tasks.py                # the Celery task
    ├── serializers.py          # model → JSON
    ├── views.py                # the API view + cache
    ├── urls.py                 # flights/
    └── management/commands/
        └── fetch_flights.py    # python manage.py fetch_flights
```

🧠 **The service layer**

All the real work lives in `services.py`. The **command** (`fetch_flights`) and the **Celery task** both just call `fetch_all_prices()`. So the code is written **once**, and can be run by hand *or* by the schedule.

Git history

DATE	COMMIT
Oct 22	first commit · docker, env, settings setup is done · comands, services
Oct 23	Add flight models, Travelpayouts service and fetch_flights command
Oct 27	Fetch and save cheapest flights for next 3 months
Oct 27	Add Celery task and daily Beat schedule for fetching flights
Oct 27	Add flights API endpoint with Redis cache
Oct 28	Send daily Telegram message with cheapest flights
Oct 28	Hide Telegram token in errors and add explanation comments

💡 **Commit message tip**

The Oct 23 commit message accidentally includes extra pasted text ("⭐ my pick, it says everything…"). Copy only the message itself. To fix the **last** commit message (before pushing): `git commit --amend -m "New message"`.

3 Step 1: Setup

GOAL A clean project folder with a virtual environment, secrets in `.env`, and Postgres + Redis running in Docker.

What we did

1. Created a virtual environment `.venv` and installed the packages.
2. Wrote `.gitignore` (so `.venv/`, `.env`, `__pycache__/` etc. never go to GitHub).
3. Saved the packages with `pip freeze > requirements.txt`, then ran `git init` and connected the GitHub repo.
4. Wrote `.env` with the database values and the API token.
5. Wrote `docker-compose.yml` with two services: `db` and `redis`.
6. Ran `django-admin startproject config .` (the dot = "create it *here*").
7. Connected `settings.py` to `.env` and to Postgres.

Key code

.ENV (PLACEHOLDERS)

```
POSTGRES_DB=flights
POSTGRES_USER=flights_user
POSTGRES_PASSWORD=<SECRET>
DB_HOST=localhost
DB_PORT=5434
TRAVELPAYOUTS_TOKEN=<SECRET>
TELEGRAM_BOT_TOKEN=<SECRET>
TELEGRAM_CHAT_ID=<YOUR_CHAT_ID>
```

DOCKER-COMPOSE.YML

```
services:
  db:
    image: postgres:17
    env_file:
      - .env
    ports:
      - "5434:5432"
    volumes:
      -
postgres_data:/var/lib/postgresql/data
  redis:
    image: redis:7
    ports:
      - "6379:6379"

volumes:
  postgres_data:
```

SETTINGS.PY: READING .ENV

```
import os
from dotenv import load_dotenv

BASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")      # a separate line, not "BASE_DIR = load_dotenv(...)"

DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ.get("POSTGRES_DB"),
        "USER": os.environ.get("POSTGRES_USER"),
        "PASSWORD": os.environ.get("POSTGRES_PASSWORD"),
        "HOST": os.environ.get("DB_HOST"),
        "PORT": os.environ.get("DB_PORT"),
    }
}
```

🧠 **Port mapping "host:container"**

`"5434:5432"` means: port **5434 on my Mac** → port **5432 inside the container**. Postgres always listens on 5432 inside, but on the Mac I can choose any free port. Django runs on the Mac, so it uses `localhost:5434`.

🧠 **Volumes and the first init**

The volume `postgres_data` keeps the database files, so the data survives a restart. But Postgres reads `POSTGRES_*` **only the first time** the volume is created. If you change the password later, the old one is still inside. Then you must delete the volume (`down -v`), which deletes all the data too.

Mistakes & fixes

PROBLEM	WHY IT HAPPENED	FIX
A git repo in my **home folder** (`~/.git`)	An old `git init` in the wrong place, so every file looked "untracked"	Deleted `~/.git` (after checking it had no commits)
`BASE_DIR = load_dotenv(...)` → NameError, then BASE_DIR became `True`	`load_dotenv` returns True/False, not a path	Keep `BASE_DIR = Path(...)`, and call `load_dotenv(...)` on its own line
Settings used `DB_NAME`, .env had `POSTGRES_DB`	Names must match exactly	Use the `POSTGRES_*` names (Docker needs them) + `DB_HOST` / `DB_PORT`
"role flights_user does not exist"	Port 5432 was used by a **local Mac Postgres**, so Django talked to the wrong database	Found it with `lsof -i :5432`, then moved to another port
Port 5433: password auth failed	5433 was used by another container (`personal_site_db`)	Moved to **5434**
Port 5434: password auth failed	The volume was created earlier with an **old password**	`docker compose down -v` + `up -d` (the DB was empty, so it was safe)

PROBLEM	WHY IT HAPPENED	FIX
YAML errors in docker-compose	"services" written twice, wrong `ports` format, redis under `volumes`, `"5432:6379"`	Indentation matters in YAML; each service has its own block; Redis is `"6379:6379"`

4 Step 2: Models & admin

GOAL Two tables: **ROUTE** (what to check) and **FLIGHTOFFER** (what we found). Both visible in the admin.

```python
class Route(models.Model):
    origin = models.CharField(max_length=3)         # BSZ
    destination = models.CharField(max_length=3)    # TYO
    is_active = models.BooleanField(default=True)

    def __str__(self):
        return f"{self.origin} - {self.destination}"


class FlightOffer(models.Model):
    route = models.ForeignKey(Route, on_delete=models.CASCADE)
    departure_at = models.DateTimeField()
    price = models.DecimalField(max_digits=10, decimal_places=2)
    airline = models.CharField(max_length=3)
    transfers = models.PositiveSmallIntegerField()
    duration = models.PositiveSmallIntegerField()
    link = models.TextField()
    found_at = models.DateTimeField(auto_now_add=True)
```

```python
# admin.py: two ways to register
admin.site.register(Route)                  # simple, default look

@admin.register(FlightOffer)                # decorator + own settings
class FlightOfferAdmin(admin.ModelAdmin):
    list_display = ["route", "departure_at", "price", "airline", "transfers"]
```

🧠 **ForeignKey + CASCADE**

Each offer **belongs to** one route. `CASCADE` = if the route is deleted, its offers are deleted too.

🧠 **auto_now_add**

Django fills `found_at` with "now" automatically when the row is **created**. That's good for `found_at`, but wrong for `departure_at` (the API gives us that).

🧠 **The model workflow**

Change `models.py` → `makemigrations` (write the plan) → `migrate` (apply it to the database). Then `createsuperuser` to log into `/admin/`, where I added the 2 routes.

Mistakes & fixes

PROBLEM	WHY IT HAPPENED	FIX
`departure_at` had `auto_now_add=True`	Copied from `found_at`	Plain `DateTimeField()`, because the value comes from the API
No `airline` field	Forgot it	Added `CharField(max_length=3)`
`self.self` in `__str__`	Typo	`self.origin`
Link didn't fit `CharField(500)`	Aviasales links are very long (~370 chars + domain)	`TextField()` = no length limit
`admin.register(Route)` did nothing	Missing `.site`, and the decorator wasn't used on a class	`admin.site.register(Route)` / `@admin.register(...)` above a class

5 Step 3: Fetching prices

GOAL Ask Travelpayouts for flights, keep the 3 cheapest with max 1 stop, and save them. Run it with `python manage.py fetch_flights`.

Three functions, three jobs

```
fetch_all_prices()          for each route × each month:
   └── save_prices()            filter + keep 3 + save
         └── get_prices()           one API request → list of dicts
```

GET_PRICES: ONE API CALL

```python
def get_prices(origin, destination, month):
    token = os.environ.get("TRAVELPAYOUTS_TOKEN")
    response = requests.get(
        API_URL,
        params={
            "origin": origin, "destination": destination,
            "departure_at": month,          # "2026-11" = the whole month
            "one_way": "true", "currency": "usd",
            "sorting": "price", "token": token,
        },
        timeout=20,
    )
    response.raise_for_status()
    return response.json()["data"]
```

SAVE_PRICES: FILTER, SLICE, SAVE

```python
def save_prices(route, month):
    flights = get_prices(route.origin, route.destination, month)
    good_flights = [f for f in flights if f["transfers"] < 2]   # max 1 stop
    three_first_flights = good_flights[:3]                          # 3 cheapest
    for t in three_first_flights:
        FlightOffer.objects.create(route=route, departure_at=t["departure_at"], …,
                                   link="https://www.aviasales.com" + t["link"])
```

FETCH_ALL_PRICES: THE NEXT 3 MONTHS

```
today = date.today()
months = []
for i in range(1, 4):              # 1, 2, 3 → starting from NEXT month
    m = today.month + i
    y = today.year
    if m > 12:                     # 13 → January of next year
        m = m - 12
        y = y + 1
    months.append(f"{y:02d}-{m:02d}")   # ["2026-11", "2026-12", "2027-01"]
```

🧠 **List comprehension**

`[x for x in items if condition]` = a new list with only the items that pass. Like `[a for a in ages if a >= 18]`. `f` is **one dict**, so we compare `f["transfers"]`, not `f`.

🧠 **Why range(1, 4)?**

`range` stops **before** the end: 1, 2, 3. We start at 1 because we want **next** month, not this one. The `if m > 12` works like a clock: 13 o'clock → 1.

🧠 **Management commands**

A file in `flights/management/commands/` becomes `python manage.py <file name>`. Django looks for a class named exactly `Command(BaseCommand)` with a `handle(self, *args, **options)` method. (`*args` = values without names, `**options` = values with names. Django passes them, so we must accept them.) Both folders need an `__init__.py`.

🧠 **Reading API documentation**

The docs are the main source: they say the URL, the parameters, and the **keys** in the answer (`data`, `transfers`, `price`, `link` ...). `response.json()` turns the text into a Python dict; `result["data"]` is the list of flights; `data[0]` is the first flight.

Mistakes & fixes

PROBLEM	WHY IT HAPPENED	FIX
400: "unknown location code FRU"	Travelpayouts uses **BSZ** for Bishkek (FRU is the old airport code)	Found BSZ with the autocomplete API; changed the routes in the admin
`if f < 2` → TypeError	`f` is a whole dict; you can't compare a dict with a number	`f["transfers"] < 2`
`flights[0]["transfers"]` in the filter	That always checks the **first** flight, not each one	Use the loop variable `f`
`[:3]` put on `f["transfers"]`	Slicing belongs to the **list**, not to one value	A separate line: `good_flights[:3]`
"has no attribute 'Command'"	The file was **not saved**	Cmd+S 🙂
`today.month = ...` failed	`date` objects can't be changed (immutable)	Use new variables `m` and `y`
`y = today.year + i`	The year shouldn't grow every month	`y = today.year`, and +1 only when `m > 12`

PROBLEM	WHY IT HAPPENED	FIX
`today.y` → AttributeError; `str(y, m)` → TypeError	No such attribute; `str()` takes one value	An f-string: `f"{y:02d}-{m:02d}"`

6 Step 4: Celery & Beat

GOAL Run `fetch_all_prices()` in the background, automatically, every day at 9:00 Bishkek time.

🧠 **The three roles**

- **Beat** ⏰ = the clock. At 9:00 it sends a message: "run this task".
- **Broker** 📨 = Redis (drawer 0). Messages wait here.
- **Worker** 👷 = takes the message and runs the Python code.

CONFIG/CELERY.PY

```
import os
from celery import Celery

os.environ.setdefault(
    "DJANGO_SETTINGS_MODULE",
    "config.settings")

app = Celery("config")
app.config_from_object(
    "django.conf:settings",
    namespace="CELERY")
app.autodiscover_tasks()
```

CONFIG/__INIT__.PY

```
from .celery import app as celery_app
__all__ = ("celery_app",)
```

FLIGHTS/TASKS.PY

```
from celery import shared_task
from .services import (fetch_all_prices,
    build_message, post_telegram)

@shared_task
def fetch_flights_task():
    fetch_all_prices()
    post_telegram(build_message())
```

SETTINGS.PY

```
CELERY_BROKER_URL =
"redis://localhost:6379/0"
CELERY_TIMEZONE = "Asia/Bishkek"

CELERY_BEAT_SCHEDULE = {
  "every_morning_fetch_flights": {
    "task":
"flights.tasks.fetch_flights_task",
    "schedule": crontab(hour=9, minute=00),
  },
}
```

🧠 **namespace="CELERY"**

Celery reads only the settings that start with `CELERY_`. So `CELERY_BROKER_URL` becomes Celery's `broker_url`, `CELERY_TIMEZONE` becomes `timezone`, and so on.

⚠️ **When to restart what**

Python loads the code **once**, when the program starts.
Changed **tasks / services code** → restart the **worker**.
Changed the **schedule** → restart **Beat**.
Changed **views / urls** → `runserver` restarts by itself.

Mistakes & fixes

PROBLEM	WHY IT HAPPENED	FIX
Worker: `ValueError: not enough values to unpack (expected 3, got 0)`	Celery's default **prefork** pool doesn't work well with macOS + Python 3.14	Start the worker with `--pool=solo`
`crontab(hour=09)` → SyntaxError	Python doesn't allow numbers with a leading zero	`hour=9`
The task never ran	The `"task"` path was copied from an example	The real path: `flights.tasks.fetch_flights_task` (app.file.function)

> 💡 **Testing a schedule quickly**
>
> Use `crontab()` with nothing inside = **every minute**. Then switch back to `hour=9` and restart Beat. To run the task once by hand: `fetch_flights_task.delay()` in the Django shell.

7 Step 5: API & Redis cache

GOAL `GET /api/flights/` returns the newest day's offers, cheapest first, as JSON. Redis caches the answer for 1 hour.

SERIALIZERS.PY

```
class
RouteSerializer(serializers.ModelSerializer):
    class Meta:
        model = Route
        fields = ["origin", "destination"]

class FlightOfferSerializer(
        serializers.ModelSerializer):
    route = RouteSerializer()   # nested

    class Meta:
        model = FlightOffer
        fields = ["id", "route",
"departure_at",
                  "price", …, "found_at"]
```

URLS

```
# config/urls.py
path("api/", include("flights.urls")),

# flights/urls.py
urlpatterns = [
  path("flights/",
       FlightOfferListView.as_view())
]
# → /api/flights/
```

JSON RESULT

```
{ "id": 85,
  "route": {"origin": "BSZ",
            "destination": "TYO"},
  "price": "397.00", … }
```

VIEWS.PY

```python
@method_decorator(cache_page(60 * 60), name="get")
class FlightOfferListView(ListAPIView):
    serializer_class = FlightOfferSerializer

    def get_queryset(self):
        if FlightOffer.objects.exists():
            newest = FlightOffer.objects.latest("found_at")
            return FlightOffer.objects.filter(
                found_at__date=timezone.localtime(newest.found_at).date()
            ).order_by("price")
        else:
            return FlightOffer.objects.none()
```

🧠 **Reading get_queryset**

- `exists()` → True/False: "is there any row?" (`latest()` crashes on an empty table)
- `latest("found_at")` → **one** object, the newest. We only need it to know **the day**.
- `filter(found_at__date=…)` → **all** offers **on** that day (`=` = exactly; `__gte` would be "that day and later")
- `order_by("price")` → cheapest first (`"-price"` = most expensive first)
- `none()` → an empty QuerySet → the API shows `[]` with status 200. That's not an error, there's just nothing yet.

🧠 **`__` vs `.`**

Inside `filter(...)` → double underscore, a message to the database: `found_at__date` , `price__lt=300` , `route__origin="BSZ"` .
Anywhere else (on an object you already have) → dots: `newest.found_at.date()` .

The cache

```
 1st request:  browser → Django → PostgreSQL → JSON → copy saved in Redis (drawer 1) → browser
 next 1 hour:  browser → Django → copy found in Redis ✨ → browser    (no database)
 after 1 hour: the copy expires (TTL) → next request goes to PostgreSQL again
 9:00 fetch:   cache.clear() → old copy deleted → the next request gets fresh prices
```

```python
# settings.py
CACHES = {"default": {
    "BACKEND": "django.core.cache.backends.redis.RedisCache",
    "LOCATION": "redis://localhost:6379/1",     # drawer 1 (Celery uses 0)
}}

# services.py, at the end of fetch_all_prices(), after both loops
cache.clear()
```

🧠 **Lazy caching**

The copy is made only when someone **opens** the API. Celery never fills the cache; it only **empties** it.

🧠 **Why clear AND expire?**

The copy expires after 1 hour anyway. `clear()` is for the case where a copy from 8:30 is still alive at 9:00, when new prices arrive.

Mistakes & fixes

PROBLEM	WHY IT HAPPENED	FIX
`include("flights.urls.py")`	`include` takes a Python **module path**, not a file name	`include("flights.urls")`
`FlightOfferListView()` in urls	A class view must be turned into a function	`.as_view()`
AttributeError: 'list' has no 'values' / missing Meta.model	RouteSerializer had no `class Meta`, then `model: Route` (colon)	`class Meta:` with `model = Route`
`found_at__at`, `.date` without `()`	No `__at` lookup; a method needs `()` to run	`found_at__date=… .date()`
`if not ():` / `return []`	`()` is an empty tuple (always "not"), and `[]` is a list, not a QuerySet	`else: return FlightOffer.objects.none()`
Wrong import: `from django.views.decorators import cache`	That's the decorators module (where `cache_page` lives), not the cache	`from django.core.cache import cache`
`cache.clear()` planned inside `save_prices`	That runs 6 times, and someone could cache a half-updated list	Once, at the end of `fetch_all_prices`

8 Step 6: Telegram bot

GOAL After the 9:00 fetch, send a nicely formatted message with all offers to my Telegram.

Setting up the bot

1. In Telegram: **@BotFather** → `/newbot` → name *Bishkek Tokyo Flights* → username *Nurbek_flights_bot*.
2. BotFather gives a **token** → put it in `.env` as `TELEGRAM_BOT_TOKEN`.
3. Send "Hi" to the bot (a bot can't write to you first).
4. Open `…/bot<TOKEN>/getUpdates` → find `"chat": {"id": …}` → put it in `.env` as `TELEGRAM_CHAT_ID`.

🧠 **Token vs chat ID** ✉️

Like a letter: the **token** is your signature (which bot, and proof you may use it). The **chat ID** is the address (who gets it). The URL start is always the same; the last word is the **method**: `getMe` (who am I?), `getUpdates` (what did people send me?), `sendMessage` (send this).

POST_TELEGRAM (FINAL VERSION)

```
def post_telegram(text):
    token = os.environ.get("TELEGRAM_BOT_TOKEN")
    chat_id = os.environ.get("TELEGRAM_CHAT_ID")
    try:
        response = requests.post(
            f"https://api.telegram.org/bot{token}/sendMessage",
            data={"chat_id": chat_id, "text": text, "parse_mode": "HTML"},
            timeout=20,
        )
    except requests.RequestException:
        raise RuntimeError("Telegram: could not connect") from None
    if not response.ok:
        raise RuntimeError(f"Telegram error: {response.json()['description']}")
```

BUILD_MESSAGE: TWO LISTS, ONE MESSAGE

```
def build_message():
    newest = FlightOffer.objects.latest("found_at")
    day = timezone.localtime(newest.found_at).date()
    # BIG list = the whole message
    lines = [f"✈️ Cheapest flights ({day.strftime('%b %d')})", ""]

    for active_route in Route.objects.filter(is_active=True):
        offers = FlightOffer.objects.filter(
            route=active_route, found_at__date=day
        ).order_by("departure_at")

        # SMALL list = one box for this route
        route_lines = [f"<b>{active_route.origin} → {active_route.destination}</b>"]
        month = None                                # the "sticky note"
        zone = AIRPORT_ZONES[active_route.origin]

        for offer in offers:
            departure = timezone.localtime(offer.departure_at, zone)
            if month is not None and departure.month != month:
                route_lines.append("")              # empty line between months
            month = departure.month

            if offer.transfers == 0:
                status = "direct"
            else:
                status = "1 stop"

            link = offer.link.split("?")[0]           # (inline in the real file)
            route_lines.append(
                f"{departure.strftime('%b %d')} ${offer.price:.0f} "
                f"{offer.airline} {status} <a href='{link}'>Book</a>"
            )

        lines.append("<blockquote>" + "\n".join(route_lines) + "</blockquote>")

    return "\n".join(lines)
```

(Shortened a little for the page. The real file is in `flights/services.py`.)

🧠 "\n".join(list)

Glue all items of a list into **one string**, with a new line between them. `", ".join(["a","b"])` → `"a, b"`. Used twice: once per route (inside the box) and once for the whole message.

🧠 The sticky note 📝

`month` remembers the **previous** offer's month. If the new one is different, add `""` (an empty line). `is not None` skips the very first offer. Update the note **every** time.

🧠 Formatting

`strftime("%b %d")` → `Nov 05` (`%B` full month, `%Y` year, `%H:%M` time).
`f"{x:.0f}"` → no numbers after the dot (it rounds).
`f"{m:02d}"` → at least 2 digits.

🧠 Telegram HTML

`<b>…</b>` bold · `<a href='…'>Book</a>` link behind a word · `<blockquote>` a box. They only work with `"parse_mode": "HTML"`.

Mistakes & fixes

PROBLEM	WHY IT HAPPENED	FIX
`os.environ.post(...)`	Mixed up with `requests.post`	Reading is always `os.environ.get`
`{"chat_id"=chat_id}`	Dicts use `:` ; `=` is for variables and arguments	`{"chat_id": chat_id}`
`/getUpdates` used for sending	getUpdates **reads** messages	`/sendMessage`
`"parse_mod"`: no error, but no HTML	Telegram **silently ignores** unknown keys	`"parse_mode"`. Names must match the docs exactly
Browser link → 404	`bot` was missing before the token	`…/bot<TOKEN>/getUpdates`
Token visible in the chat (twice)	Pasted a link; then the `raise_for_status()` error printed the full URL	BotFather → `/revoke` → new token in .env. Then the code fix in Step 7
400: **text is too long**	34,512 characters (limit is **4,096**): each link was 369 chars + 98 offers	`link.split("?")[0]` keeps only the short part
`day.objects.filter`	`.objects` exists only on **models**; `day` is just a date	`FlightOffer.objects.filter(..., found_at__date=day)`
`newest.date`, `newest.found_at__date()`	Mixed up the filter style and the object style	`newest.found_at.date()`
`return` with 8 spaces	It was inside the loop, so the function stopped after the first route	4 spaces: after **all** loops
`"... status"` printed the word "status"	Only things in `{ }` are variables in an f-string	`{status}`
Offers outside the box	`lines.append` instead of `route_lines.append`	Offers go into the small list

9 Step 7: Timezones & security

GOAL Show the right day and the right departure dates, and never leak the token in an error message.

One moment, three clocks



Problem 1: the wrong "newest day"

I ran the fetch at **00:19 Bishkek time on Oct 28**, which was still **Oct 27 in UTC**. With `TIME_ZONE = "UTC"`, Django thought the newest day was Oct 27, so it mixed in all the test batches from that day: **98 offers** with repeats instead of **14**.

```
# settings.py
TIME_ZONE = "Asia/Bishkek"      # left side: found_at__date now cuts days on the Bishkek clock

# views.py and services.py
day = timezone.localtime(newest.found_at).date()   # right side: UTC → Bishkek, then the date
```

> ⚠️ **The trap**
>
> Even with `TIME_ZONE = "Asia/Bishkek"`, an object from the database still has its times in **UTC**. So `newest.found_at.date()` would still be the UTC date. Both sides of the comparison must use **the same clock**, which is why we need `timezone.localtime()`.

Problem 2: a departure date off by one day

A Tokyo flight showed **Dec 13**, but its link said `TY01412` (**Dec 14**). A ticket shows the local time of the **departure airport**, so each route needs its own clock:

```
AIRPORT_ZONES = {
    "BSZ": ZoneInfo("Asia/Bishkek"),
    "TYO": ZoneInfo("Asia/Tokyo"),
}
zone = AIRPORT_ZONES[active_route.origin]
departure = timezone.localtime(offer.departure_at, zone)    # 2nd argument = which clock
```

> 🧠 **Two times, two jobs**
>
> `found_at` = when **we** saved it → **our** clock (Bishkek). It decides *which* offers to show.
> `departure_at` = when the **plane** leaves → the **airport's** clock. It decides *how* the date looks.
> The database itself always stores UTC (`USE_TZ = True`), and that's good.

Hiding the token in errors 🔒

❌ `before:  HTTPError: 400 Client Error: Bad Request for url: https://api.telegram.org/bot<TOKEN!>/sendMessage`
✅ `after:  RuntimeError: Telegram error: Bad Request: message text is empty`

🧠 **try / except / raise ... from None**

- `try:` → try to run this code
- `except requests.RequestException:` → if any `requests` error happens (no internet, timeout…)
- `raise RuntimeError("…")` → make **our own** error with our message
- `from None` → hide the original error (its message contains the URL with the token)
- `response.ok` → True for 2xx, False for 4xx/5xx. Telegram's reason is in `response.json()["description"]`

💡 **Security habits I learned**

- Secrets live only in `.env`, and `.env` is in `.gitignore`. Check `git status` before every commit.
- Never paste a token in a chat, issue or screenshot. If it happens: **revoke it right away** (BotFather `/revoke`).
- Error messages and logs can contain URLs, so don't put secrets in URLs you print.
- To test a token safely: `curl …/bot$TELEGRAM_BOT_TOKEN/getMe` after `source .env`, so the terminal fills it in and you never see it.

10 Concepts glossary

Python

CONCEPT	MEANING · EXAMPLE
venv	A private folder of packages for one project (`.venv`). Activate it before working.
dict / key	`{"price": 405}` → `d["price"]`. JSON becomes dicts and lists.
list index / slice	`items[0]` = first · `items[:3]` = first three
list comprehension	`[f for f in flights if f["transfers"] < 2]`
f-string formats	`{m:02d}` → 01 · `{p:.0f}` → 405 · only `{ }` parts are variables
`join` / `split`	`"\n".join(lines)` list → text · `link.split("?")[0]` text → list → first piece
`strftime`	Date → text in any format: `d.strftime("%b %d")` → Nov 05
`None` / `is not None`	"No value yet". Compare with `is`, not `==`.
`return`	Give a value back and **stop** the function. Indentation decides where.
`*args` / `**options`	Extra values without names (a tuple) / with names (a dict)

CONCEPT	MEANING · EXAMPLE
decorator `@`	Wraps a function/class with extra behaviour: `@shared_task`, `@admin.register`, `@method_decorator`
`try` / `except` / `raise`	Catch an error · make your own error
`timeout`	Max seconds to wait for an answer. Without it, a request can wait forever.

Django & DRF

CONCEPT	MEANING · EXAMPLE
QuerySet	Works like a list, but it's really a database query. The **database** filters and sorts.
`filter` / `order_by`	`.filter(route=r, found_at__date=day).order_by("price")` · chain from left to right
`latest` / `exists` / `none`	One newest object · True/False · an empty QuerySet
lookups `__`	`__date`, `__lt`, `__gte`, `__year`, `route__origin`. Only inside `filter()`.
`.objects`	Exists only on **model classes** (`FlightOffer.objects`), not on values
migrations	`makemigrations` = write the plan · `migrate` = apply it
management command	Our own `manage.py` command: `class Command(BaseCommand)` + `handle()`
serializer	Model ↔ JSON. Nested serializer = show a related object inside.
`ListAPIView` / `get_queryset`	A ready view that returns a list · override it to choose **which** rows
`.as_view()`	Turns a class view into a function that URLs can call
`include()`	Use another app's `urls.py` under a prefix (`"api/"`)
TIME_ZONE / USE_TZ / localtime	Store UTC, think in Bishkek, convert with `timezone.localtime(dt, zone)`

Infrastructure

CONCEPT	MEANING · EXAMPLE
container / image	An image = a recipe (`postgres:17`) · a container = a running copy of it
port mapping	`"5434:5432"` = Mac port → container port
volume	Where a container keeps data that must survive restarts
broker	Where tasks wait for the worker (Redis drawer 0)
worker / Beat	Runs tasks · a clock that starts tasks on a schedule
`.delay()`	"Worker, please run this task" (through the broker)
crontab	A schedule: `crontab(hour=9, minute=0)` = every day 9:00
cache / TTL	A saved copy of an answer · "time to live" in seconds (–2 = key doesn't exist)
Redis databases	"Drawers" 0–15 in one Redis: `/0` Celery, `/1` cache

CONCEPT	MEANING · EXAMPLE
HTTP status	200 OK · 400 bad request · 401 unauthorized (bad token) · 404 not found · 500 server error
GET / POST	GET = ask for data · POST = send data

11 Lessons learned & next ideas

Lessons learned

1. **Read the error message slowly.** "role does not exist" was really a port problem. "text is too long" told me exactly what to fix. Telegram's `description` was much more useful than "400".
2. **Print first, then build.** Printing routes and offers before making the message made each step easy to check.
3. **Indentation is logic.** The same line at 4 or 8 spaces means "once" or "every time".
4. **Names must match exactly:** env variable names, `parse_mode`, task paths, API keys. Some systems give no error at all when a name is wrong.
5. **Timezones:** store UTC, show local. Decide which clock each date belongs to.
6. **One job per function:** `get_prices` → `save_prices` → `fetch_all_prices`; the task adds Telegram, so the command doesn't spam me.
7. **Secrets:** `.env` + `.gitignore`, check `git status`, and revoke anything that leaks.
8. **Restart after code changes:** the worker keeps the old code until it restarts.
9. **Destructive commands** (`down -v`, `rm -rf`, deleting rows): look at the target first, then decide.

Ideas for next time

IDEA	WHAT I'D LEARN
Tests (`pytest`, Django `TestCase`)	Mocking the APIs, testing `build_message` and the month logic
Price alerts	Only message me when a price drops below a limit, or is lower than yesterday
Price history chart	Aggregations (`Min`, `Avg`) per day, a small frontend
More routes from the admin	Add routes without code; put the timezone in the `Route` model instead of a dict
Retries for API calls	Celery `autoretry_for`, `retry_backoff`
Run Django + Celery in Docker too	A Dockerfile, so `docker compose up` starts everything
Deploy to a server	No more "the Mac must be awake at 9:00" 🙂
Clean old offers	A weekly task that deletes offers older than N days
Filters in the API	`?route=BSZ-TYO`, `?max_price=400` with `django-filter`

🎉 **What I built**

A real scheduled backend: an external API → a database → a cached REST API → a daily notification, with Docker, Redis in two roles, Celery + Beat, timezones and basic security. That's a full small system!

12 Terminal cheat sheet

Run these from the project folder `~/Desktop/cheap_flight_checker`. **Red** = can delete data, so think first.

🐍 Virtual environment & pip

COMMAND	WHAT IT DOES
`python3 -m venv .venv`	Create a virtual environment in `.venv`
`source .venv/bin/activate`	Turn it on (you'll see `(.venv)`)
`deactivate`	Turn it off
`pip install django djangorestframework "psycopg[binary]" python-dotenv requests celery redis`	Install the project packages
`pip freeze > requirements.txt`	Save the exact package versions
`pip install -r requirements.txt`	Install everything from the file (new computer)
`which python`	Check that you're using the `.venv` Python

🎸 Django

COMMAND	WHAT IT DOES
`django-admin startproject config .`	New project **in this folder** (the dot!)
`python manage.py startapp flights`	New app
`python manage.py runserver`	Start the web server → http://127.0.0.1:8000
`python manage.py makemigrations`	Write migration files from model changes
`python manage.py migrate`	Apply migrations to the database
`python manage.py createsuperuser`	Make an admin login
`python manage.py shell`	Python with Django loaded (`exit()` to leave)
`python manage.py shell -c "…"`	Run one piece of Python code and exit
`python manage.py check`	Look for problems in the project
`python manage.py fetch_flights`	Our command: fetch + save + clear the cache (no Telegram)
`python manage.py help fetch_flights`	Show a command's help text

🐋 Docker Compose

COMMAND	WHAT IT DOES
`docker compose up -d`	Start Postgres + Redis in the background

COMMAND	WHAT IT DOES
`docker compose ps`	Show which containers are running
`docker compose logs db`	Show a container's logs (add `-f` to follow)
`docker compose down`	Stop and remove the containers (the data stays in the volume)
`docker compose down -v`	**Also delete the volumes: ALL database data is gone**
`docker compose exec redis redis-cli`	Open the Redis terminal inside the container
`docker ps`	All running containers on the Mac (also other projects)

🧰 Redis CLI

COMMAND	WHAT IT DOES
`docker compose exec redis redis-cli -n 1 KEYS "*"`	List all keys in drawer 1 (the cache)
`docker compose exec redis redis-cli -n 1 TTL "<key>"`	Seconds left (–1 = never expires, –2 = doesn't exist)
`docker compose exec redis redis-cli -n 0 KEYS "*"`	Look at drawer 0 (Celery)
`docker compose exec redis redis-cli PING`	Is Redis alive? → `PONG`

🌿 Celery

COMMAND	WHAT IT DOES
`celery -A config worker -l info --pool=solo`	Start the worker (`--pool=solo` is needed on macOS + Python 3.14)
`celery -A config beat -l info`	Start Beat (the 9:00 clock). Needs its own terminal.
`Ctrl + C`	Stop the worker / Beat (then start again to load new code)
`ps aux | grep celery`	Check if the worker and Beat are running

`-A config` = the Celery app is in the `config` package · `-l info` = log level (how much to print).

🌱 Git

COMMAND	WHAT IT DOES
`git init`	Start a repo in this folder
`git status`	What changed? **Check that .env is NOT listed**
`git add .`	Stage all changes (.gitignore files are skipped)
`git commit -m "Message"`	Save a snapshot

COMMAND	WHAT IT DOES
`git commit --amend -m "New message"`	Fix the last commit's message (before pushing)
`git remote add origin <url>`	Connect to GitHub
`git push`	Upload commits to GitHub
`git log --oneline`	Short commit history
`git diff`	See the exact line changes before committing

📱 Telegram

COMMAND	WHAT IT DOES
`@BotFather → /newbot`	Create a bot and get a token
`@BotFather → /revoke`	Kill a leaked token and get a new one
`set -a; source .env; set +a`	Load .env into the terminal (then use `$TELEGRAM_BOT_TOKEN`)
`curl -s "https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN/getMe"`	Test the token (shows the bot's name)
`curl -s "https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN/getUpdates"`	Messages people sent → find your chat ID

🍎 Mac helpers

COMMAND	WHAT IT DOES
`lsof -i :5432`	Which program is using a port?
`pmset -g`	Show sleep and power settings (`sleep 0` = never sleeps)
`caffeinate -i`	Keep the Mac awake while it runs (Ctrl+C to stop)
`nohup caffeinate -i > /dev/null 2>&1 &`	Same, in the background (survives closing the terminal)
`pkill caffeinate`	Stop caffeinate
`rm -rf <path>`	**Delete a folder forever. Double-check the path!**
`clear`	Clean the terminal screen

13 Shell snippets & daily checklist

Open with `python manage.py shell`. After changing code, `exit()` and open it again; the shell loads the code only once.

RUN THE MORNING TASK BY HAND

```
from flights.tasks import fetch_flights_task
fetch_flights_task.delay()
# → watch the worker terminal + phone
```

SEND / PREVIEW THE MESSAGE

```
from flights.services import (
    build_message, post_telegram)

print(build_message())          # preview
post_telegram(build_message())  # send
post_telegram("Hello ✈️")        # test
```

CACHE

```
from django.core.cache import cache
cache.set("hello", "world", 60)  # 60 sec
cache.get("hello")               # 'world'
cache.clear()                    # empty
drawer 1
```

LOOK AT THE DATA

```
from flights.models import Route, FlightOffer
Route.objects.exists()
FlightOffer.objects.count()
FlightOffer.objects.latest("found_at")
FlightOffer.objects.filter(
    route__origin="BSZ").order_by("price")
[:3]
```

✅ Before 9:00 checklist

CHECK	HOW
🔌 Mac on the charger	On battery it sleeps after 1 minute
💻 Lid open	A closed lid always sleeps (unless an external monitor is connected)
☕ Optional extra safety	`nohup caffeinate -i > /dev/null 2>&1 &` · stop later with `pkill caffeinate`
🐳 Docker running	`docker compose ps` → db + redis "running"
👷 Worker running (new code)	`celery -A config worker -l info --pool=solo`
⏰ Beat running	`celery -A config beat -l info`
🖥️ Terminals stay open	Closing a terminal stops its program

🔧 No message at 9:00? Troubleshooting

WHAT YOU SEE	LIKELY CAUSE	WHAT TO DO
Nothing in the worker terminal	Beat isn't running, or the Mac was asleep / restarted	Start Beat; check `ps aux | grep celery`
Worker: connection refused (Redis / Postgres)	Docker isn't running	`docker compose up -d`
`RuntimeError: Telegram: could not connect`	No internet at that moment	Run `fetch_flights_task.delay()` again later
`RuntimeError: Telegram error: …`	Telegram refused (bad token, text too long…)	Read the description; test with `getMe`
HTTPError 400/401 from Travelpayouts	Bad token or a wrong parameter	Check `TRAVELPAYOUTS_TOKEN` and the city codes

WHAT YOU SEE	LIKELY CAUSE	WHAT TO DO
Old prices in the API	The cache wasn't cleared, or the worker has old code	Restart the worker; `cache.clear()`
ValueError: not enough values to unpack	Worker started without `--pool=solo`	Restart with `--pool=solo`